CSC2626: Assignment 2
22 points

Setup

Unzip `A2.zip` to get the starter coder for this assignment. Follow the instructions in the `README.md` file contained under A2 folder to set up the python environment and install the dependencies. *We recommend running this assignment on macOS or Linux, as there can be issues with the* `mujoco-py` *package on Windows.* Please post any issue regarding the setup on Piazza or send the TA an email.

1 Conservative Q-Learning (6 points)

The goal of this question is to help you get familiar with offline reinforcement learning, and in particular with the implementation of Conservative Q-Learning [Kumar et al., 2020].

Provide a summary, up to 2 pages, of the main ideas and assumptions in the paper. Ensure that you provide a description of the main equations and theorems and describe the significance of the experimental results with respect to related baseline methods. In addition, make sure to address the following questions in your summary, by looking at the reference implementation.[1]

- Which equation in the paper was used to implement the CQL update rule in the reference implementation?
- How was this equation derived in the paper?
- How is the equation above implemented for discrete systems and continuous systems?
- Do the CQL lower bounds hold upon convergence of the CQL update rule, or also during each Q iteration?
- How is the policy learned? Is there another way it could have been computed?
- (Bonus Question, 1pt) What directions for future work would you consider in order to improve, extend, or make use of this method or the lower bounds that it guarantees?

2 Dynamic Movement Primitives (9 points)

In this question, you will use Dynamic Movement Primitives (DMPs) to learn a policy which enables a 6-DOF robot arm (UR10) to catch a ball. See Figure 1. You will be provided with a set of demonstration trajectories collected from an expert policy, which you will use to train the parameters of the DMP.

2.1 Implementing Dynamic Movement Primitives (6 points)

You will be implementing the version of DMPs that is described in Pastor et al. [2009].

1. **Learning a DMP**: Fill in the missing code for the learning procedure `DMP.learn` in `dmp.py`. This instance method takes as input a set of trajectories and solves a least squares problem to set the weight parameters of a DMP.

   The inputs to this method are two numpy arrays, one with shape (N, T, D) and the second with shape (N, T) where N is the number of demonstrations, T is the number of timesteps, and D is the number of degrees of freedom. Note: that each degree of freedom is to be treated as a different DMP with a different set of weights, but all of them share the same phase variable.

2. **Querying a DMP:** Fill in the code for the planning procedure `DMP.execute` in `dmp.py`.

[1] https://github.com/aviralkumar2907/CQL/blob/master/d4rl/rlkit/torch/sac/cql.py

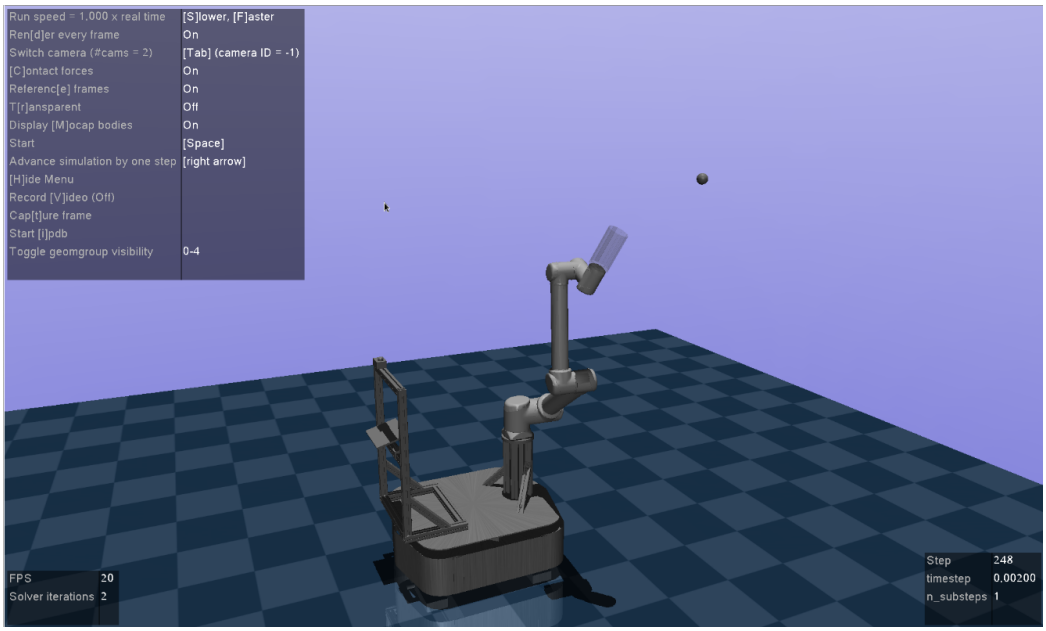


Figure 1: A screenshot from the `ballcatch-v0` environment. Courtesy of Shichen Lu and Ke Dong.

2.2 Using Dynamic Movement Primitives (3 points)

You will be working with a small dataset of expert demonstrations provided under `data/demos.pkl`. Each element i in the dataset is a numpy array of shape $(T_i, 6)$, where T_i is the number of timesteps in trajectory i. For each timestep, the demo contains the joint angles for the 6DOF robot in figure 1. Note that each timestep corresponds to a duration of 0.04 seconds.

1. **Interpolation of Variable Length Trajectories**: Each trajectory in the dataset has a different length T_i. Fill in the code in `dmp._interpolate` in `dmp.py` to perform a linear interpolation of the trajectories such that all of them will have the same length. Note: you can use `scipy._interpolate` for this.

2. **Reconstruction using DMP**: Use your implementation from above to train a DMP **only** on the first demonstration in the dataset (i.e. $i = 0$). (For this question, keep all the hyperparameters in their default setting.) Then try to reconstruct the trajectory. You can use `DMP.rollout` function for this, setting the initial position x_0 to the first timestep of the trajectory, and the goal to the final timestep. Visualize the reconstruction by plotting the demo trajectory and reconstruction for each degree of freedom separately. You will find boilerplate for this in `q2_recons()` in `main.py`. To run your code, use `python main.py recons`

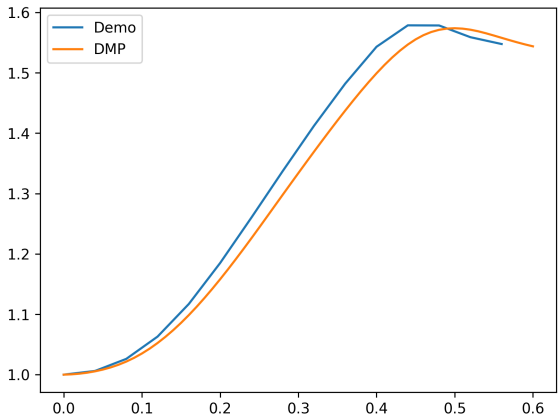


Figure 2: DMP reconstruction of demo0 degree of freedom 0.

3. **Tuning**: Train a DMP on the full set of trajectories. Experiment with different settings of the number of basis functions `DMP.nbasis` and the gain parameter `DMP.K_vec`. You can evaluate the fit quantitatively

using RMSE against the demos. Once you have determined appropriate settings, fill in the relevant code under `q2_tuning()` in `main.py`. Finally, run `python main.py tuning`. Note: This will save your DMP to disk, so it can be used in the remaining questions.

4. **Closed Loop Policy**: So far, we have used the DMP to plan in open loop trajectories. This assumes that the robot's motion is perfect, and ignores the effects of inertia, torque limits, etc. When we try to execute the policy on a real (or in this case, simulated) robot, we may end up veering off the planned trajectory. DMP provides a natural way of representing closed loop policies, which correct themselves based on the position at the current time. Fill in the missing lines in `BallCatchDMPPolicy.select_action` in `policy.py` to do this.

5. **Catch some balls**: Now, you will plug your DMP into the Mujoco simulator for ball catching, and see how well it does. To do this, simply run `python main.py`. This should launch the mujoco simulator, and use your saved DMP and closed loop policy to drive the arm. Report the average reward and success rate across 20 episodes. Record a video of the results to submit along with your report.

3 Diffusion Policy (7 points)

In this problem, you'll be implementing Diffusion Policy Chi et al. [2025] which has become the dominant policy class in imitation learning. Notably, a variant of this is used for Vision-Language-Action models Black et al. [2024] which are arguably at the frontier of generalist robotics.

3.1 Implement DDPM (4 points)

The first part of the assignment will have you implementing the DDPM algorithm you'll use to train your diffusion policy. We give you a neural network backbone and all the boilerplate code needed to run it on a Robomimic environment.

For help implementing this, it may be useful to refer to the original DDPM paper Ho et al. [2020], these two references on diffusion models Nakkiran et al. [2024], Chan et al. [2024], or the Diffusion Policy paper Chi et al. [2025]

All of the following steps should be done in the ddpm.py file.

1. **Implement Noise Addition** Finish implementing add_noise(), which generates a noisy target from a clean data sample.

2. **Implement Loss** Finish implementing loss(), which uses the function you just implemented to generate the learning target for the diffusion model. Note that you are asked to implement both noise prediction and clean sample prediction - the notation in Ho et al. [2020] uses epsilon-prediction, but the two reference works discuss the natural extension to clean sample prediction. These are typically called "epsilon prediction" and "x prediction" or "x0 prediction". Which is better often depends on other parts of the diffusion formulation, so one isn't universally better than another. You'll try both for this class.

3. **Implement Sampling** Finally, implement sample(), which will let you generate samples from your score-predicting neural network.

3.2 Implement Action Chunking (2 Points)

The other important component of diffusion policy besides diffusion itself is *action chunking* Chi et al. [2025], Zhao et al. [2023]. Action chunking breaks the Markovian assumption of the environment by modeling a series of actions given an observation. A traditional policy only predicts one action $\pi(o) \rightarrow a$, while action chunking predicts several $\pi(o) \rightarrow [a_1, a_2, ...a_H]$ of which it executes the first k open-loop before quereying the policy again. Implement this change in `diffusion_policy.py` (see around line 121).

3.3 Run Diffusion Policy (1 Points)

If everything is implemented correctly, you should be able to run `python diffusion_policy.py` to train diffusion policy with noise prediction, and `python diffusion_policy.py --predict_x0` to train it with clean prediction. As a baseline, also run `bc.py` which will train a standard behaviour cloning policy, without action chunking or diffusion. **Note**: it will likely take 8 hours total to run all of these, so give yourself plenty of time. Now you will analyze the results.

1. **Plot Completion vs Training Steps** run `plot_completion_vs_gradient_steps.py`, which should generate a plot showing completion rate as a function of the training gradient steps.

2. **Plot DDPM Sampling** run `python plot_diffusion_generation.py` and `python plot_diffusion_generation.py --predict_x0` which will generate videos of how the sample changes during the reverse diffusion process. Take a look at these.

3.4 Bonus (1 Points)

For a bonus point, experiment with one substantial algorithmic modification to diffusion policy, run it with the same parameters as before, and report on your findings. Examples could include modifying some diffusion modeling parameter (such as a new noise schedule), changing the action space somehow, or using an alterate generative modeling objective (such as flow-matching). Include a short write-up in your submission as `bonus.pdf`. Discuss how performance differed and what you found surprising.

4 What/How to submit

Submit a file called `assignment_firstname_lastname_studentid.zip` on Quercus. This should contain the following:

- A file called `cql.pdf`, containing your summary of CQL.
- `dmp.py`
- `main.py`
- `policy.py`
- `recons[1-6].png` - the visualization from the reconstructed trajectory.
- `ballcatch.mp4` - the video from the final question in the report.
- `ddpm.py`
- `diffusion_policy.py`
- `completion_vs_gradient_steps.png` - visualization of training progress for BC, Diffusion-eps and Diffusion-x0

References

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33:1179–1191, 2020.

Peter Pastor, Heiko Hoffmann, Tamim Asfour, and Stefan Schaal. Learning and generalization of motor skills by learning from demonstration. In *2009 IEEE International Conference on Robotics and Automation*, pages 763–768, 2009. doi: 10.1109/ROBOT.2009.5152385.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Preetum Nakkiran, Arwen Bradley, Hattie Zhou, and Madhu Advani. Step-by-step diffusion: An elementary tutorial. *CoRR*, abs/2406.08929, 2024. URL https://doi.org/10.48550/arXiv.2406.08929.

Stanley Chan et al. Tutorial on diffusion models for imaging and vision. *Foundations and Trends® in Computer Graphics and Vision*, 16(4):322–471, 2024.

Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.